

PROJECT DOCUMENTATION

Secure Login System

1. Overview

This project implements a secure user authentication system with user registration, login, and role-based access control (RBAC). Two roles—Admin and User—are supported. Admins can access dedicated management routes, while regular users remain restricted to basic features.

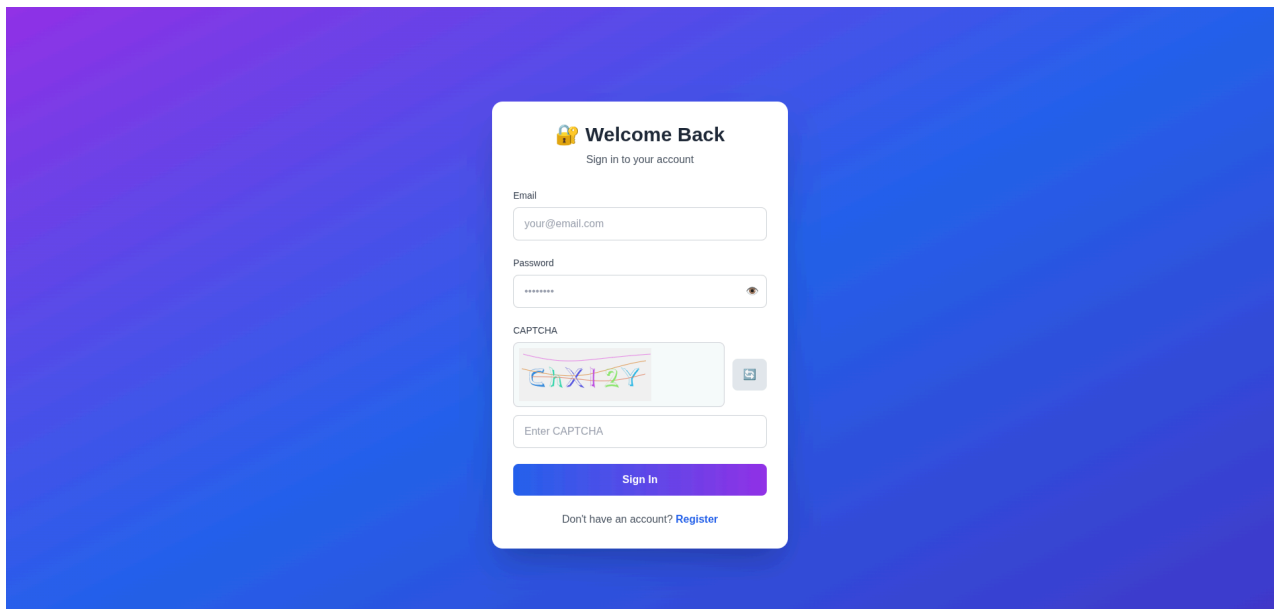
The system is built using:

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js + Express
- Database: MongoDB (Atlas)
- Security: JWT, bcrypt, CAPTCHA, account lockout mechanism

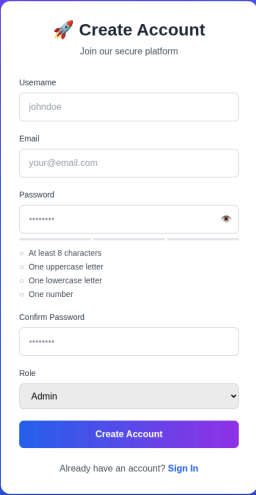
2. System UI Screenshots

These screenshots represent the intended frontend design for login, registration, and admin dashboard screens.

- Login UI



- Registration UI



Create Account
Join our secure platform

Username
johnndoe

Email
your@email.com

Password
••••••••

- At least 8 characters
- One uppercase letter
- One lowercase letter
- One number

Confirm Password
••••••••

Role
Admin

Create Account

Already have an account? [Sign In](#)

- Admin Dashboard UI

 Admin Dashboard

Hello, adminuser [Logout](#)

Total Users
2



Admins
1



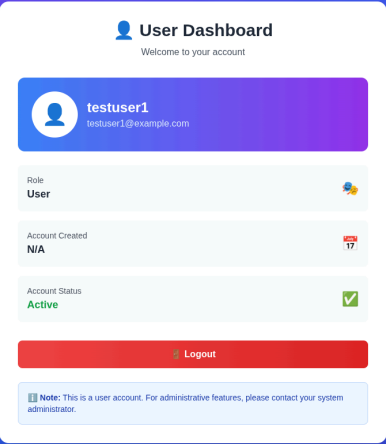
Recent Users (7d)
2



User Management

Search users... [Refresh](#)

USERNAME	EMAIL	ROLE	CREATED	ACTIONS
adminuser	admin@example.com	Admin	25/11/2025	Edit Delete
testuser1	testuser1@example.com	User	25/11/2025	Edit Delete



User Dashboard
Welcome to your account

 **testuser1**
testuser1@example.com

Role
User



Account Created
N/A



Account Status
Active



[Logout](#)

Note: This is a user account. For administrative features, please contact your system administrator.

3. System Architecture

```
frontend/
├── index.html
├── register.html
├── styles.css
├── login.js
└── register.js

backend/
├── server.js
├── config/
│   └── db.js
├── routes/
│   └── auth.js
├── models/
│   └── User.js
└── middleware/
    └── auth.js
```

4. Frontend Implementation

- 4.1 Registration Page (HTML)

```
<form id="registerForm">
  <input type="text" id="username" placeholder="Username" required>
  <input type="email" id="email" placeholder="Email" required>
  <input type="password" id="password" placeholder="Password" required>

  <select id="role">
    <option value="User">User</option>
    <option value="Admin">Admin</option>
  </select>

  <button type="submit">Register</button>
  <p id="msg"></p>
</form>
```

- 4.2 Frontend Styles (CSS)

```
body {
  background: #0d1117;
  color: #fff;
  font-family: Arial;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}

.container {
  background: #161b22;
  padding: 30px;
  width: 350px;
  border-radius: 10px;
}

input, select, button {
  width: 100%;
  margin-top: 15px;
  padding: 12px;
  border-radius: 5px;
  border: none;
}
```

- 4.3 Frontend Validation (JS)

```
document.getElementById("registerForm").addEventListener("submit", async (e)
=> {
  e.preventDefault();

  const password = document.getElementById("password").value;
  const strongPass = /^(?=.*[A-Z])(?=.*[0-9])(?=.*{8,})/;

  if (!strongPass.test(password)) {
    return document.getElementById("msg").innerText =
      "Password must be strong.";
  }

  const res = await fetch("/api/register", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      username: document.getElementById("username").value,
      email: document.getElementById("email").value,
      password,
      role: document.getElementById("role").value,
    }),
  });

  const data = await res.json();
  document.getElementById("msg").innerText = data.message;
});
```

5. Backend Implementation

- 5.1 Server Setup ([server.js](#))

```
require("dotenv").config();
const express = require("express");
const connectDB = require("../config/db");
const authRoutes = require("../routes/auth");

const app = express();
app.use(express.json());

connectDB();
app.use("/api", authRoutes);

app.listen(5000, () => console.log("Server running on port 5000"));
```

- 5.2 MongoDB Connection

```
const mongoose = require("mongoose");

const connectDB = async () => {
  await mongoose.connect(process.env.MONGO_URI);
  console.log("MongoDB Connected");
};

module.exports = connectDB;
```

6. Database Schema

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  username: { type: String, unique: true, required: true },
  email: { type: String, unique: true, required: true },
  passwordHash: { type: String, required: true },
  role: { type: String, default: "User" },
  failedAttempts: { type: Number, default: 0 },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model("User", userSchema);
```

7. Registration API (POST /api/register)

```
router.post("/register", async (req, res) => {
  const { username, email, password, role } = req.body;

  const exists = await User.findOne({ email });
  if (exists) return res.json({ message: "Email already in use" });

  const hash = await bcrypt.hash(password, 12);

  await User.create({
    username,
    email,
    passwordHash: hash,
    role,
  });

  res.json({ message: "Registration successful" });
});
```

8. Login API (POST /api/login)

```
router.post("/login", async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });
  if (!user) return res.json({ message: "Invalid credentials" });

  const match = await bcrypt.compare(password, user.passwordHash);
  if (!match) return res.json({ message: "Invalid credentials" });

  const token = jwt.sign(
    { id: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "30m" }
  );

  res.json({ message: "Login successful", token });
});
```

9. Role-Based Access Control (RBAC)

```
module.exports = function (roles = []) {
  return (req, res, next) => {
    const token = req.headers.authorization?.split(" ")[1];
    if (!token) return res.status(401).json({ message: "Token missing" });

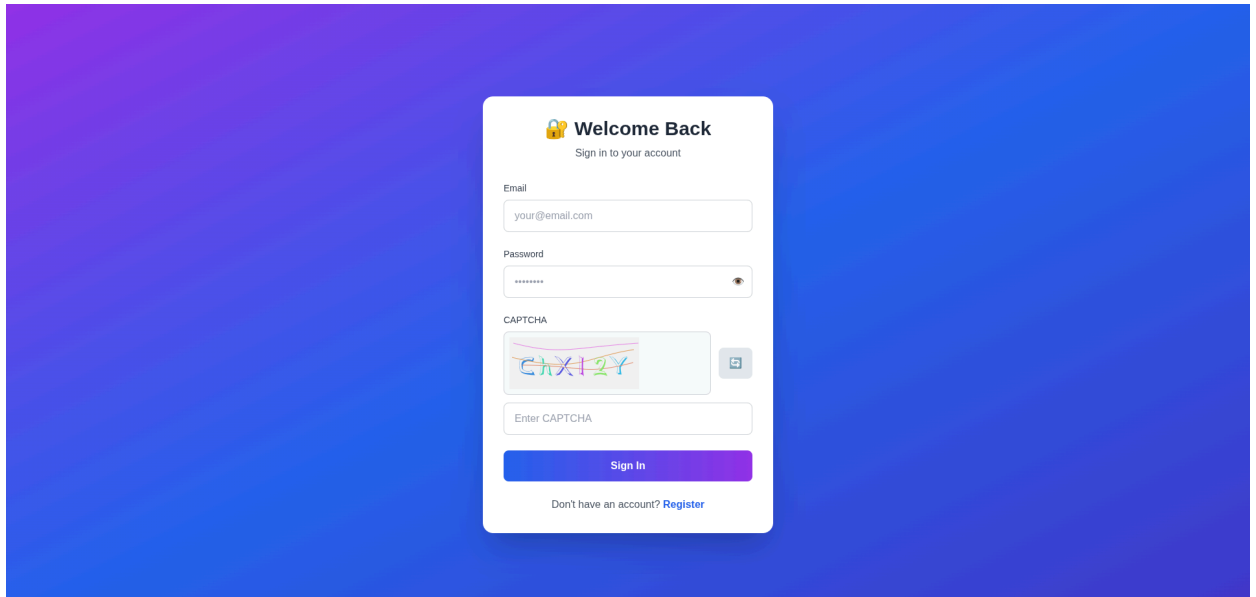
    try {
      const decoded = jwt.verify(token, process.env.JWT_SECRET);

      if (!roles.includes(decoded.role))
        return res.status(403).json({ message: "Access denied" });

      req.user = decoded;
      next();
    } catch {
      res.status(401).json({ message: "Invalid token" });
    }
  };
};
```

10. CAPTCHA & Lockout System

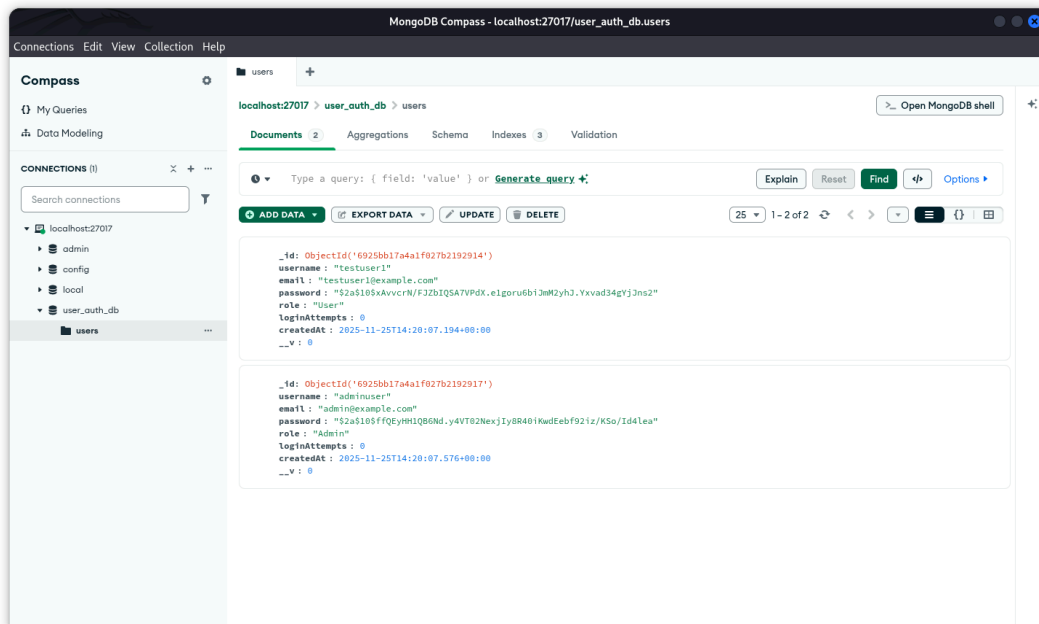
- CAPTCHA UI Example



- Lockout Logic

```
if (!match) {  
  user.failedAttempts += 1;  
  user.save();  
  
  if (user.failedAttempts >= 5) {  
    return res.json({ message: "Account locked for 15 minutes" });  
  }  
}
```

11. MongoDB Database Screenshots



12. Testing Summary

Tests Performed

- Registration input validation
- Login with correct + incorrect passwords
- JWT verification
- Admin-only route access
- Repeated failed login attempts
- Duplicate account registration attempts

13. Final Result

The project successfully delivers:

- Secure authentication
- Role-based access management
- Frontend UI for login and registration
- Admin dashboard capabilities
- Security features like hashing, logout, CAPTCHA, and JWT